

Álgebra Superior con aplicaciones en R y GeoGebra

Teoría, visualización y experimentación

MI Jesús Gilberto Rodríguez Escobedo

7 de agosto de 2026

Índice

Ruta de aprendizaje	3
1 Lógica y conjuntos	4
Introducción	4
Objetivos	4
1.1 Lenguaje de conjuntos	4
Situación inicial	4
Conjunto potencia	5
1.2 Proposiciones y conectivos lógicos	6
Construcción de una tabla de verdad	6
Tautologías, contradicciones y contingencias	6
Interactivo: tabla de verdad	7
1.3 Cuantificadores y equivalencias lógicas	7
Proposiciones abiertas sobre dominios finitos	7
Negación de cuantificadores	8
1.4 Operaciones y propiedades de los conjuntos	8
Operaciones en R	8
Interactivo: operaciones de conjuntos	9
Comprobación de las leyes de De Morgan	10
1.5 Cardinalidad de conjuntos finitos y problemas de conteo	10
Inclusión-exclusión con dos conjuntos	10
Inclusión-exclusión con tres conjuntos	10
Interpretación	11
1.6 Producto cartesiano y relaciones	11
Producto cartesiano como tabla	11
Comprobación de una identidad	11
Propiedades de una relación finita	12
1.7 Funciones: definición, representación y operaciones	13
Una función como tabla	13
Funciones dadas por fórmulas	14
1.8 Funciones inyectivas, suprayectivas y biyectivas	15
Clasificación automática de funciones finitas	15
Interactivo: clasificador de funciones	15
1.9 Cardinalidad, equipotencia y conjuntos infinitos	16
Una biyección entre N_0 y Z	16
Aplicación integradora: álgebra de Boole y circuitos	17
Compuertas básicas en R	17
De circuito a expresión y de expresión a circuito	17
Comprobación de una simplificación	18
Universalidad de NOR y NAND	18
Señales activas en alto y en bajo	19

Formas de onda lógicas	20
Laboratorio del capítulo	20
Ejercicios con R	21
Proyecto integrador	21
Recursos interactivos y multimedia	21
Síntesis	22
Referencias del capítulo	22

Este libro propone aprender Álgebra Superior mediante tres formas complementarias de trabajo:

1. resolución manual para comprender el procedimiento;
2. experimentación reproducible en R;
3. visualización dinámica en GeoGebra.

Los laboratorios integrados y el cuaderno de Google Colab permiten modificar datos, formular conjeturas y comprobar resultados. El código se explica paso a paso y no sustituye al razonamiento matemático: lo hace visible, verificable y extensible.

La primera versión incluye interactivos autónomos para operaciones con conjuntos, tablas de verdad y clasificación de funciones. Funcionan directamente en el libro web publicado mediante GitHub Pages.

Ruta de aprendizaje

Cada sección sigue, cuando corresponde, esta secuencia:

- situación inicial;
- ideas matemáticas indispensables;
- solución manual;
- implementación en R sin paquetes;
- visualización o construcción en GeoGebra;
- laboratorio interactivo;
- interpretación y ejercicios.

1 Lógica y conjuntos

Experimentación con R, GeoGebra y circuitos

Introducción

Este capítulo transforma los conceptos de lógica y conjuntos en objetos que pueden calcularse, visualizarse y comprobarse. Cada bloque parte de una idea matemática, la resuelve manualmente y después utiliza R como laboratorio. GeoGebra se reservará para las representaciones dinámicas y Shiny para actividades en las que el estudiante modifique datos y reciba retroalimentación.

El código inicial utiliza R base (R Core Team 2026). De esta forma puede ejecutarse en RStudio o en el cuaderno de Google Colab sin instalar paquetes.

El cuaderno reúne las nueve secciones, las aplicaciones a sistemas digitales, visualizaciones en R y ejercicios de comprobación. Puede abrirse en Colab o descargarse para conservar una copia local.

[Abrir en Google Colab](#) [Descargar el cuaderno](#)

Objetivos

Al finalizar, el lector podrá:

- representar conjuntos finitos mediante vectores de R;
- construir tablas de verdad;
- evaluar proposiciones abiertas sobre dominios finitos;
- comprobar equivalencias lógicas y leyes de conjuntos;
- resolver problemas de inclusión-exclusión;
- representar productos cartesianos, relaciones y funciones;
- clasificar funciones finitas;
- experimentar con correspondencias biyectivas;
- comprobar simplificaciones booleanas y diseñar circuitos.

1.1 Lenguaje de conjuntos

Situación inicial

Consideremos el universo

$$U = \{1, 2, \dots, 10\},$$

el conjunto A de números pares y el conjunto B de números primos en U :

$$A = \{2, 4, 6, 8, 10\}, \quad B = \{2, 3, 5, 7\}.$$

En R, un conjunto finito puede representarse mediante un vector sin elementos repetidos.

```
U <- 1:10
A <- c(2, 4, 6, 8, 10)
B <- c(2, 3, 5, 7)
```

```
A
B
```

El operador `%in%` comprueba pertenencia.

```
4 %in% A
5 %in% A
A %in% B
```

Para comprobar $A \subseteq B$, no basta que algún elemento pertenezca; todos deben pertenecer:

```
es_subconjunto <- function(X, Y) {
  all(X %in% Y)
}
```

```
es_subconjunto(c(2, 4), A)
es_subconjunto(A, B)
```

La igualdad de conjuntos debe ignorar el orden:

```
setequal(c(1, 2, 3), c(3, 2, 1))
identical(c(1, 2, 3), c(3, 2, 1))
```

La función `setequal()` devuelve verdadero, mientras que `identical()` devuelve falso, porque esta última compara vectores ordenados, no conjuntos.

Conjunto potencia

El siguiente código genera todos los subconjuntos de un conjunto finito:

```
conjunto_potencia <- function(A) {
  resultado <- list(A[FALSE])
  if (length(A) > 0) {
    for (k in seq_len(length(A))) {
      resultado <- c(
        resultado,
        combn(A, k, simplify = FALSE)
      )
    }
  }
}
```

```

      resultado
    }

P <- conjunto_potencia(c("a", "b", "c"))
P
length(P)

```

El resultado contiene $2^3 = 8$ subconjuntos, incluido el conjunto vacío.

Se construirá una mesa dinámica con los ocho subconjuntos de $\{a, b, c\}$. El estudiante podrá seleccionar dos subconjuntos y observar su unión, intersección y diferencia simétrica. El enlace se incorporará cuando la actividad esté publicada.

1.2 Proposiciones y conectivos lógicos

Construcción de una tabla de verdad

R representa verdadero y falso mediante **TRUE** y **FALSE**. Las combinaciones para dos proposiciones se generan así:

```

tabla <- expand.grid(
  q = c(TRUE, FALSE),
  p = c(TRUE, FALSE)
)[, c("p", "q")]

implica <- function(p, q) {
  (!p) | q
}

tabla$no_p <- !tabla$p
tabla$p_y_q <- tabla$p & tabla$q
tabla$p_o_q <- tabla$p | tabla$q
tabla$p_implica_q <- implica(tabla$p, tabla$q)
tabla$p_sii_q <- tabla$p == tabla$q

```

tabla

La implicación se implementa mediante la equivalencia

$$p \Rightarrow q \equiv \neg p \vee q.$$

Tautologías, contradicciones y contingencias

Una expresión es tautología si todas sus filas son verdaderas:

```

es_tautologia <- function(valores) all(valores)
es_contradiccio n <- function(valores) all(!valores)

tercero_excluido <- tabla$p | !tabla$p
contradiccio n <- tabla$p & !tabla$p

es_tautologia(tercero_excluido)
es_contradiccio n(contradiccio n)

```

Para comprobar

$$p \Rightarrow (q \wedge r) \equiv (p \Rightarrow q) \wedge (p \Rightarrow r),$$

generamos las ocho combinaciones posibles:

```

t3 <- expand.grid(
  r = c(TRUE, FALSE),
  q = c(TRUE, FALSE),
  p = c(TRUE, FALSE)
)[, c("p", "q", "r")]

izquierda <- implica(t3$p, t3$q & t3$r)
derecha <- implica(t3$p, t3$q) & implica(t3$p, t3$r)

data.frame(t3, izquierda, derecha, equivalentes = izquierda == derecha)
all(izquierda == derecha)

```

La última instrucción devuelve verdadero; la equivalencia se cumple en todas las filas.

Interactivo: tabla de verdad

Cambie la expresión y los valores de entrada. La fila activa se señala en la tabla y la salida se actualiza inmediatamente.

[Abrir el laboratorio de tablas de verdad](#)

1.3 Cuantificadores y equivalencias lógicas

Proposiciones abiertas sobre dominios finitos

Sea

$$D = \{-4, -3, \dots, 4\}$$

y sea $p(x) : x^2 \leq 4$. Su conjunto solución se obtiene filtrando el dominio:

```
D <- -4:4
p <- D^2 <= 4

data.frame(x = D, p_x = p)
D[p]
```

El conjunto solución es $\{-2, -1, 0, 1, 2\}$.

En un dominio finito, **all()** representa el cuantificador universal y **any()** el existencial:

```
all(D^2 >= 0) # para todo x en D, x^2 >= 0
any(D^2 == 4) # existe x en D tal que x^2 = 4
```

Verificar una propiedad en muchos valores no demuestra que se cumpla para todos los números reales. R sí puede decidir un cuantificador sobre un dominio finito completamente enumerado. Para dominios infinitos, la demostración matemática sigue siendo indispensable.

Negación de cuantificadores

La equivalencia

$$\neg(\forall x \in D p(x)) \equiv \exists x \in D \neg p(x)$$

se comprueba con:

```
lado_izquierdo <- !all(p)
lado_derecho <- any(!p)

c(lado_izquierdo, lado_derecho)
```

No estamos sustituyendo una demostración formal; estamos observando la equivalencia en el dominio elegido.

1.4 Operaciones y propiedades de los conjuntos

Operaciones en R

R incluye funciones directas para unión, intersección y diferencia.

```
union(A, B)
intersect(A, B)
setdiff(A, B)
setdiff(B, A)
```

El complemento depende del universo:

```
complemento <- function(X, U) {
  setdiff(U, X)
}
```

```
complemento(A, U)
```

La diferencia simétrica es

$$A \triangle B = (A \setminus B) \cup (B \setminus A).$$

```
diferencia_simetrica <- function(X, Y) {
  union(setdiff(X, Y), setdiff(Y, X))
}
```

```
diferencia_simetrica(A, B)
```

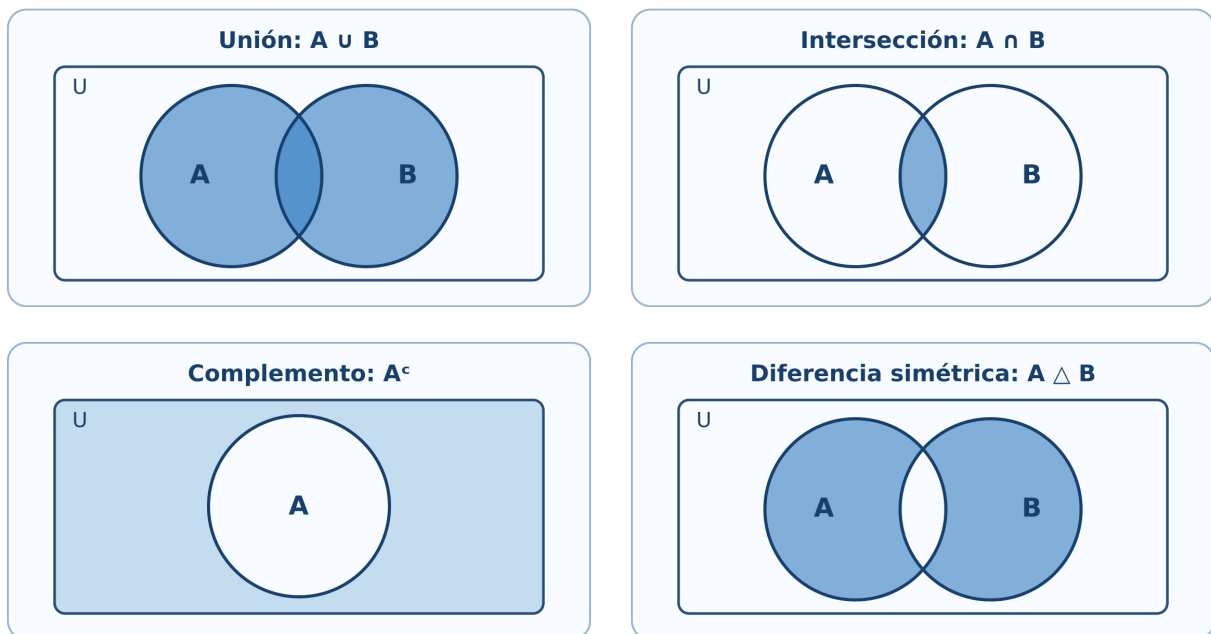


Figura 1.1: Cuatro operaciones representadas mediante regiones de Venn: unión, intersección, complemento y diferencia simétrica.

Interactivo: operaciones de conjuntos

Seleccione una operación y modifique los elementos de A y B . El laboratorio calcula el conjunto resultante, su cardinalidad y la región correspondiente.

[Abrir el laboratorio de conjuntos](#)

Comprobación de las leyes de De Morgan

Para los conjuntos concretos $A, B \subseteq U$, comprobamos

$$(A \cup B)^c = A^c \cap B^c.$$

```
lado_izquierdo <- complemento(union(A, B), U)
lado_derecho <- intersect(
  complemento(A, U),
  complemento(B, U)
)
```

```
lado_izquierdo
lado_derecho
setequal(lado_izquierdo, lado_derecho)
```

El laboratorio permite anticipar que $(A \cup B)^c$ y $A^c \cap B^c$ sombrean la misma región. La comprobación con R verifica el universo finito elegido; la demostración general se realiza mediante pertenencia.

1.5 Cardinalidad de conjuntos finitos y problemas de conteo

Inclusión-exclusión con dos conjuntos

En un grupo de 180 estudiantes, 95 usan R, 80 usan Python y 40 utilizan ambos lenguajes.

```
total <- 180
n_R <- 95
n_Python <- 80
n_ambos <- 40

n_al_menos_uno <- n_R + n_Python - n_ambos
n_ninguno <- total - n_al_menos_uno

c(al_menos_uno = n_al_menos_uno, ninguno = n_ninguno)
```

Inclusión-exclusión con tres conjuntos

```
inclusion_exclusion_3 <- function(nA, nB, nC,
                                nAB, nAC, nBC, nABC) {
  nA + nB + nC - nAB - nAC - nBC + nABC
}

total_dispositivos <- 400
al_menos_un_sensor <- inclusion_exclusion_3(
  nA = 170, nB = 150, nC = 140,
  nAB = 70, nAC = 60, nBC = 50,
  nABC = 20
)
```

)

```
ningun_sensor <- total_dispositivos - al_menos_un_sensor  
c(al_menos_un_sensor, ningun_sensor)
```

Interpretación

R no decide qué significa cada intersección. El lector debe identificar si los datos de dos conjuntos incluyen o excluyen la intersección triple. El código ejecuta correctamente una fórmula solo cuando la modelación es correcta.

1.6 Producto cartesiano y relaciones

Producto cartesiano como tabla

Sean $A = \{1, 2\}$ y $B = \{x, y, z\}$. El producto $A \times B$ se genera con `expand.grid()`:

```
A1 <- c(1, 2)  
B1 <- c("x", "y", "z")  
  
AxB <- expand.grid(a = A1, b = B1)  
AxB  
nrow(AxB)
```

El número de filas confirma $|A \times B| = |A||B| = 6$.

Comprobación de una identidad

Verifiquemos

$$A \times (B \cup C) = (A \times B) \cup (A \times C).$$

Para comparar pares ordenados utilizaremos una clave de texto.

```
clave_par <- function(x, y) paste(x, y, sep = "|")  
  
producto_cartesiano <- function(X, Y) {  
  pares <- expand.grid(x = X, y = Y)  
  clave_par(pares$x, pares$y)  
}  
  
A2 <- c(1, 2)  
B2 <- c("b1", "b2")  
C2 <- c("c1", "c2")  
  
izquierda <- producto_cartesiano(A2, union(B2, C2))  
derecha <- union(
```

```

    producto_cartesiano(A2, B2),
    producto_cartesiano(A2, C2)
)

setequal(izquierda, derecha)

```

Propiedades de una relación finita

Representemos una relación mediante dos columnas:

```

X <- c(1, 2, 3)
R <- data.frame(
  x = c(1, 1, 2, 3),
  y = c(1, 2, 2, 3)
)
R

```

Las siguientes funciones analizan reflexividad, simetría, antisimetría y transitividad.

```

claves_relacion <- function(R) clave_par(R$x, R$y)

es_reflexiva <- function(R, X) {
  all(clave_par(X, X) %in% claves_relacion(R))
}

es_simetrica <- function(R) {
  all(clave_par(R$y, R$x) %in% claves_relacion(R))
}

es_antisimetrica <- function(R) {
  reversos <- clave_par(R$y, R$x) %in% claves_relacion(R)
  all((R$x == R$y) | !reversos)
}

es_transitiva <- function(R) {
  claves <- claves_relacion(R)
  for (i in seq_len(nrow(R))) {
    for (j in seq_len(nrow(R))) {
      if (R$y[i] == R$x[j]) {
        if (!(clave_par(R$x[i], R$y[j]) %in% claves)) {
          return(FALSE)
        }
      }
    }
  }
  TRUE
}

c(

```

```

reflexiva = es_reflexiva(R, X),
simetrica = es_simetrica(R),
antisimetrica = es_antisimetrica(R),
transitiva = es_transitiva(R)
)

```

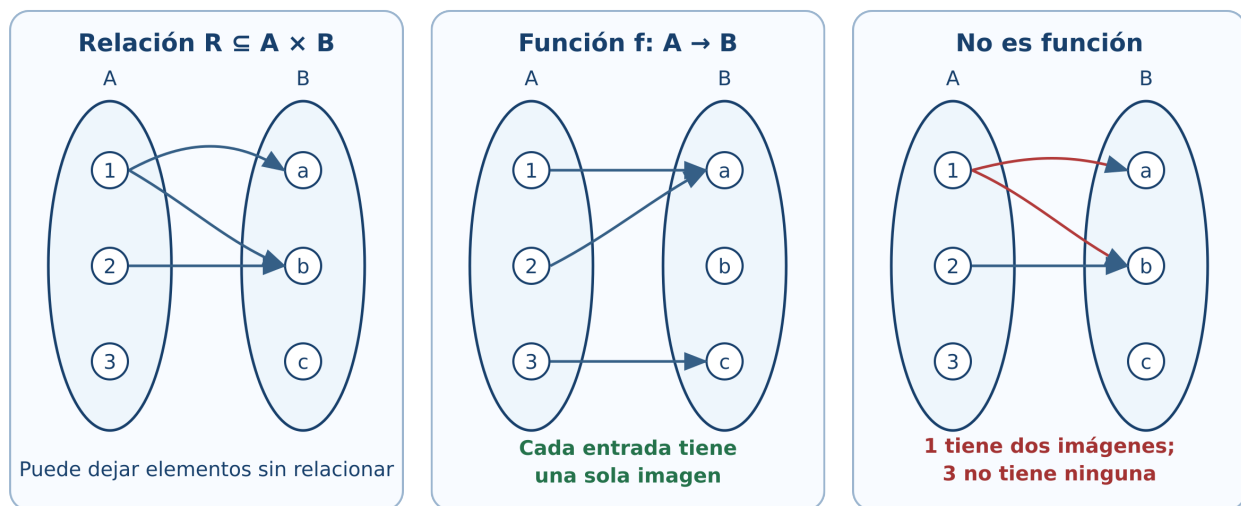


Figura 1.2: Una relación general, una función válida y una correspondencia que no cumple la definición de función.

El programa comprueba una relación finita completamente especificada. La razón matemática de cada propiedad sigue siendo la definición; el código automatiza la revisión de los pares.

1.7 Funciones: definición, representación y operaciones

Una función como tabla

Sobre conjuntos finitos, una función puede representarse por una tabla de entradas y salidas.

```

dominio <- c("s1", "s2", "s3")
codominio <- c("bajo", "medio", "alto")

```

```

f_tabla <- data.frame(
  x = dominio,
  y = c("bajo", "alto", "medio")
)

```

f_tabla

Una tabla representa una función si:

1. cada elemento del dominio aparece;
2. aparece exactamente una vez como entrada;

3. cada salida pertenece al codominio.

```
es_funcion <- function(tabla, dominio, codominio) {  
  entradas_correctas <- setequal(tabla$x, dominio)  
  entradas_unicas <- !anyDuplicated(tabla$x)  
  salidas_validas <- all(tabla$y %in% codominio)  
  
  entradas_correctas && entradas_unicas && salidas_validas  
}  
  
es_funcion(f_tabla, dominio, codominio)
```

Funciones dadas por fórmulas

```
f <- function(x) 2 * x + 1  
g <- function(x) x^2  
  
g_comp_f <- function(x) g(f(x))  
f_comp_g <- function(x) f(g(x))  
  
x <- -3:3  
data.frame(  
  x,  
  g_comp_f = g_comp_f(x),  
  f_comp_g = f_comp_g(x)  
)
```

Las columnas no coinciden: en general $g \circ f \neq f \circ g$.

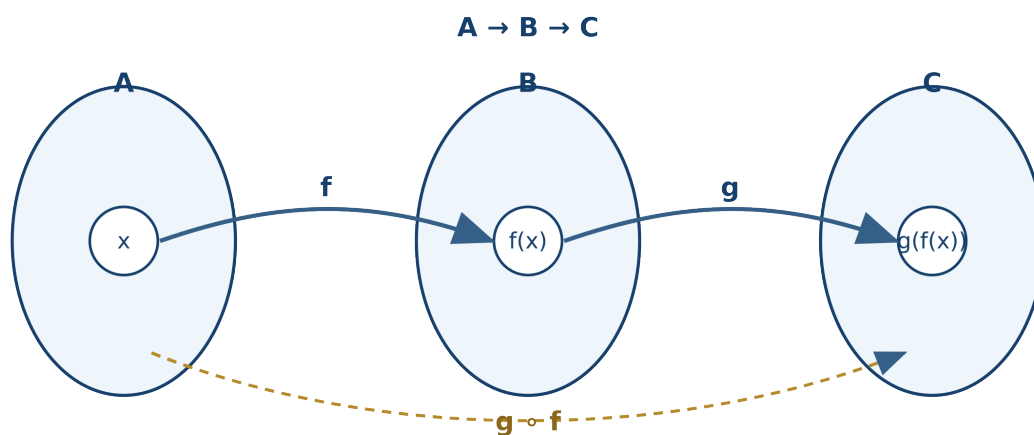


Figura 1.3: Recorrido de un elemento mediante f , después mediante g y directamente mediante la composición $g \circ f$.

Se mostrarán simultáneamente las gráficas de f , g , $g \circ f$ y $f \circ g$. Un deslizador permitirá seguir un valor x desde el dominio, pasar por la salida de la primera función y llegar a la composición.

1.8 Funciones inyectivas, suprayectivas y biyectivas

Clasificación automática de funciones finitas

```
clasificar_funcion <- function(tabla, dominio, codominio) {  
  if (!es_funcion(tabla, dominio, codominio)) {  
    return(data.frame(  
      es_funcion = FALSE,  
      inyectiva = NA,  
      suprayectiva = NA,  
      biyectiva = NA  
    ))  
  }  
  
  inyectiva <- !anyDuplicated(tabla$y)  
  suprayectiva <- setequal(unique(tabla$y), codominio)  
  
  data.frame(  
    es_funcion = TRUE,  
    inyectiva = inyectiva,  
    suprayectiva = suprayectiva,  
    biyectiva = inyectiva && suprayectiva  
  )  
}  
  
clasificar_funcion(f_tabla, dominio, codominio)
```

Probemos ahora una función que repite una salida:

```
h_tabla <- data.frame(  
  x = dominio,  
  y = c("bajo", "bajo", "medio")  
)  
  
clasificar_funcion(h_tabla, dominio, codominio)
```

No es inyectiva porque dos entradas producen “bajo”, y no es suprayectiva porque “alto” no aparece como imagen.

Interactivo: clasificador de funciones

Modifique las tres imágenes, deje alguna entrada sin imagen o asigne dos imágenes al elemento 1. El diagnóstico distingue la definición de función de las propiedades de inyectividad, suprayectividad y biyección.

[Abrir el clasificador de funciones](#)

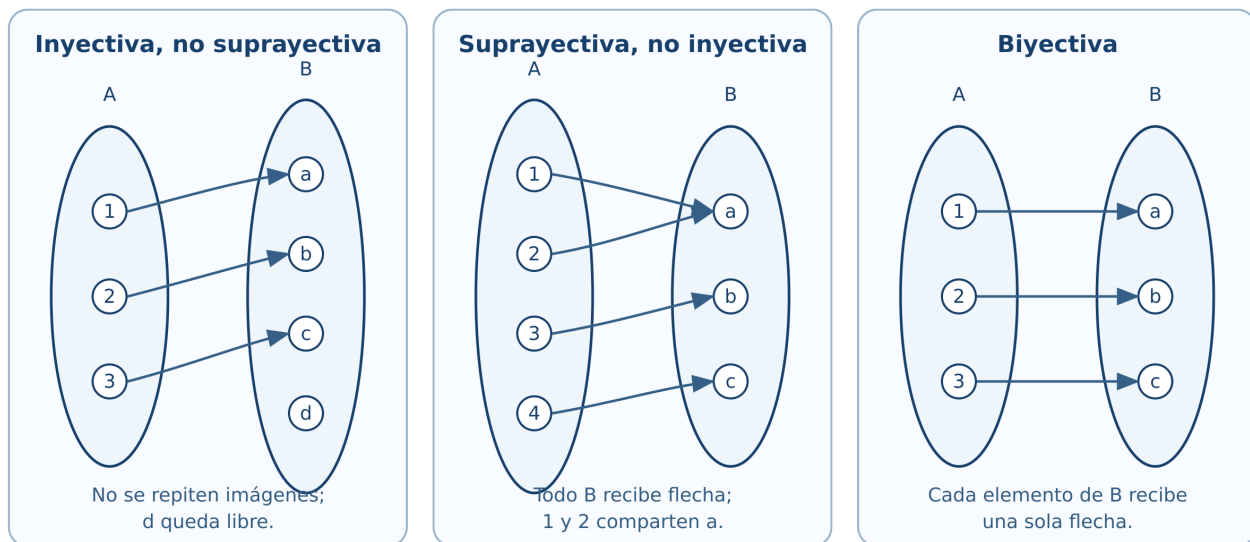


Figura 1.4: La inyectividad evita imágenes repetidas; la suprayectividad exige cubrir todo el codominio; la biyección cumple ambas condiciones.

1.9 Cardinalidad, equipotencia y conjuntos infinitos

Una biyección entre $\mathbb{N}_0$ y $\mathbb{Z}$

Definimos

$$\phi(n) = \begin{cases} 0, & n = 0, \\ (n+1)/2, & n \text{ impar}, \\ -n/2, & n \text{ par y } n > 0. \end{cases}$$

```
phi <- function(n) {
  ifelse(
    n == 0,
    0,
    ifelse(n %% 2 == 1, (n + 1) / 2, -n / 2)
  )
}
```

```
n <- 0:16
data.frame(n, phi_n = phi(n))
```

La salida comienza

0, 1, -1, 2, -2, 3, -3, ...

y muestra cómo los enteros pueden enumerarse mediante los naturales.

La tabla solo muestra los primeros valores. La biyección se justifica demostrando que cada entero aparece exactamente una vez; observar muchos casos sirve para comprender el patrón, no para sustituir la prueba.

Aplicación integradora: álgebra de Boole y circuitos

La secuencia práctica adapta el capítulo 3 de Tocci and Widmer (2003): variables booleanas, tablas de verdad, compuertas OR, AND y NOT, traducción entre circuitos y expresiones, evaluación de salidas, NAND/NOR, De Morgan, universalidad y diagramas de temporización.

Compuertas básicas en R

```
NOT <- function(a) !a
OR  <- function(a, b) a | b
AND <- function(a, b) a & b
NOR <- function(a, b) !(a | b)
NAND <- function(a, b) !(a & b)

entradas_basicas <- expand.grid(
  b = c(TRUE, FALSE),
  a = c(TRUE, FALSE)
)[, c("a", "b")]

data.frame(
  entradas_basicas,
  OR = OR(entradas_basicas$a, entradas_basicas$b),
  AND = AND(entradas_basicas$a, entradas_basicas$b),
  NOR = NOR(entradas_basicas$a, entradas_basicas$b),
  NAND = NAND(entradas_basicas$a, entradas_basicas$b)
)
```

De circuito a expresión y de expresión a circuito

Para analizar un diagrama:

1. se identifican entradas, salidas y polaridades activas;
2. se asigna una variable intermedia a la salida de cada compuerta;
3. se escribe la expresión de cada etapa de izquierda a derecha;
4. se sustituye hasta obtener la salida en función de las entradas;
5. se comprueba el resultado mediante una tabla de verdad.

Para implantar una expresión se realiza el proceso inverso. Por ejemplo,

$$X = (A + \overline{B})C$$

requiere invertir B , formar $A + \overline{B}$ con OR y combinar el resultado con C mediante AND.

El libro incorporará figuras propias con símbolos lógicos estándar. Cada ejemplo mostrará cuatro representaciones coordinadas: expresión, tabla de verdad, circuito y diagrama de temporización. No se reutilizarán las ilustraciones del texto de referencia.

Comprobación de una simplificación

Estudemos

$$F = \bar{a}b + ab + a\bar{b}.$$

Algebraicamente,

$$\begin{aligned} F &= b(\bar{a} + a) + a\bar{b} \\ &= b + a\bar{b} \\ &= (b + a)(b + \bar{b}) \\ &= a + b. \end{aligned}$$

R comprueba la igualdad en las cuatro combinaciones:

```
tv <- expand.grid(
  b = c(TRUE, FALSE),
  a = c(TRUE, FALSE)
)[, c("a", "b")]

F_original <- (!tv$a & tv$b) |
              (tv$a & tv$b) |
              (tv$a & !tv$b)

F_simplificada <- tv$a | tv$b

data.frame(
  tv,
  F_original,
  F_simplificada,
  iguales = F_original == F_simplificada
)

all(F_original == F_simplificada)
```

Universalidad de NOR y NAND

```
nor <- function(x, y) !(x | y)

no_con_nor <- function(x) nor(x, x)
o_con_nor <- function(x, y) nor(nor(x, y), nor(x, y))
y_con_nor <- function(x, y) nor(nor(x, x), nor(y, y))
```

```

entradas <- expand.grid(
  b = c(TRUE, FALSE),
  a = c(TRUE, FALSE)
)[, c("a", "b")]

data.frame(
  entradas,
  NOT_a = no_con_nor(entradas$a),
  OR_ab = o_con_nor(entradas$a, entradas$b),
  AND_ab = y_con_nor(entradas$a, entradas$b)
)

```

La misma comprobación puede realizarse con NAND:

```

nand <- function(x, y) !(x & y)

no_con_nand <- function(x) nand(x, x)
y_con_nand <- function(x, y) nand(nand(x, y), nand(x, y))
o_con_nand <- function(x, y) nand(nand(x, x), nand(y, y))

data.frame(
  entradas,
  NOT_a = no_con_nand(entradas$a),
  OR_ab = o_con_nand(entradas$a, entradas$b),
  AND_ab = y_con_nand(entradas$a, entradas$b)
)

```

Señales activas en alto y en bajo

Una señal activa en alto realiza su función cuando vale 1; una activa en bajo la realiza cuando vale 0. En un nombre de señal, la barra superior puede indicar activación en bajo. En un símbolo lógico, una burbuja indica inversión o terminal activa en bajo.

```

habilitada_alta <- c(FALSE, TRUE, FALSE, TRUE)
habilitada_baja <- !habilitada_alta

data.frame(
  nivel_entrada = as.integer(habilitada_alta),
  activa_en_alto = habilitada_alta,
  activa_en_bajo = habilitada_baja
)

```

En R, cero y uno se modelan como estados lógicos. En el dispositivo real corresponden a intervalos de voltaje definidos por su tecnología. El modelo de este capítulo no incluye todavía márgenes de ruido ni retardos de propagación.

Formas de onda lógicas

El siguiente ejemplo genera señales binarias y calcula las salidas AND y OR.

```
t <- 0:8
senal_A <- c(0, 0, 1, 1, 1, 0, 1, 1, 0)
senal_B <- c(1, 1, 1, 0, 0, 1, 0, 0, 0)

salida_AND <- as.integer(as.logical(senal_A) & as.logical(senal_B))
salida_OR <- as.integer(as.logical(senal_A) | as.logical(senal_B))

matplot(
  t,
  cbind(senal_A, senal_B, salida_AND, salida_OR),
  type = "s",
  lty = 1,
  lwd = 2,
  xlab = "Tiempo",
  ylab = "Nivel lógico",
  yaxt = "n",
  col = c("#1f77b4", "#d62728", "#2ca02c", "#9467bd")
)
axis(2, at = c(0, 1))
legend(
  "topright",
  legend = c("A", "B", "A AND B", "A OR B"),
  col = c("#1f77b4", "#d62728", "#2ca02c", "#9467bd"),
  lty = 1,
  lwd = 2,
  bty = "n"
)
```

El recurso previsto permitirá accionar interruptores para A , B y C , observar la salida del circuito y comparar el resultado con la fila correspondiente de la tabla de verdad. Otro recurso permitirá mover los instantes de conmutación y observar las formas de onda. Las compuertas se presentarán con símbolos uniformes basados en la convención IEEE/ANSI estudiada por Tocci and Widmer (2003).

Laboratorio del capítulo

La primera versión integra tres aplicaciones que funcionan directamente en el sitio estático, sin servidor ni instalación adicional:

1. **Constructor de tablas de verdad:** selección de conectivos y clasificación como tautología, contradicción o contingencia.
2. **Operaciones de conjuntos:** regiones de Venn, cardinalidad e inclusión-exclusión.
3. **Clasificador de funciones:** diagramas sagitales y diagnóstico de inyectividad, suprayectividad y biyectividad.

El **simulador de circuitos**, con compuertas, polaridades y diagramas de temporización, será la siguiente ampliación. Más adelante se preparará una versión Shiny o Shinylive cuando aporte controles que no convenga resolver en el navegador.

Ejercicios con R

1. Modifique U , A , B y compruebe ambas leyes de De Morgan.
2. Construya la tabla de verdad de $[(p \wedge \neg q) \Rightarrow (r \wedge \neg r)] \Leftrightarrow (p \Rightarrow q)$.
3. Escriba una función que determine si dos conjuntos finitos son disjuntos.
4. Extienda el programa de relaciones para indicar qué pares impiden la simetría o la transitividad.
5. Construya tres tablas: una función solo inyectiva, otra solo suprayectiva y una biyectiva.
6. Compruebe por tabla de verdad la ley de absorción $a + ab = a$.
7. Implemente NAND y demuestre computacionalmente que también es una compuerta universal.
8. Genere dos señales binarias propias y trace las salidas XOR y NOR.

Proyecto integrador

Diseñe un sistema lógico de tres entradas relacionado con sensores electrónicos:

1. describa la regla de activación;
2. escriba su expresión booleana;
3. genere la tabla de verdad en R;
4. simplifique la expresión manualmente;
5. compruebe la simplificación con R;
6. dibuje los circuitos original y simplificado;
7. genere una forma de onda de prueba;
8. interprete la reducción en el número de compuertas.

Recursos interactivos y multimedia

Esta versión ya incorpora:

- cuaderno completo del capítulo para Google Colab con R;
- laboratorio de tablas de verdad;
- laboratorio de operaciones con conjuntos y diagramas de Venn;
- clasificador de funciones mediante diagramas sagitales;
- figuras vectoriales para operaciones, relaciones, composición y tipos de funciones.

En versiones posteriores se incorporarán:

- video introductorio sobre lógica y conjuntos;
- video de tablas de verdad y cuantificadores;
- video de simplificación booleana y universalidad de NAND/NOR;
- presentación docente;
- infografía de correspondencia lógica-conjuntos;
- infografía de funciones y sus clasificaciones;

- infografía de errores frecuentes;
- actividades de GeoGebra y simulador de circuitos.

Síntesis

R permite experimentar con conjuntos finitos, tablas de verdad, relaciones y funciones. Las comprobaciones exhaustivas son concluyentes cuando el dominio es finito y se han enumerado todos los casos. Para afirmaciones generales sobre conjuntos infinitos, la demostración matemática sigue siendo el fundamento.

Referencias del capítulo

La secuencia temática sigue el programa de Álgebra Superior (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). Los fundamentos se apoyan en Silva and Lazo (2007), Epp (2011) y Rosen (2019). La aplicación a compuertas y álgebra booleana adapta el capítulo 3 de Tocci and Widmer (2003). El código utiliza R base (R Core Team 2026).

Epp, Susanna S. 2011. *Discrete Mathematics with Applications*. 4th ed. Brooks/Cole, Cengage Learning.

R Core Team. 2026. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing.

Rosen, Kenneth H. 2019. *Discrete Mathematics and Its Applications*. 8th ed. McGraw-Hill Education.

Silva, Juan Manuel, and Adriana Lazo. 2007. *Fundamentos de Matemáticas: Álgebra, Trigonometría, Geometría Analítica y Cálculo*. 7th ed. México: Limusa.

Tocci, Ronald J., and Neal S. Widmer. 2003. *Sistemas Digitales: Principios y Aplicaciones*. 8th ed. México: Pearson Educación.

Universidad Autónoma de San Luis Potosí, Facultad de Ciencias. 2011. “Programa Sintético de Álgebra Superior, Carrera de Ingeniero Electrónico.” Propuesta de modificación curricular.